

Trình Bài Làm

Artificial Intelligence
Lab 01 & Lab 02

22205801 - Huỳnh Thế Bảo (Lab 01)

22302421 - Nguyễn Phúc Minh Châu (Lab 02)

Lab 01

Use 'Lab1_dataset.csv'.

- a) Missing data could be anything in { NA, n/a, "", -1, ...}. Read the data and assign the missing data in the dataframe as **NaN**. Hint: `df.replace( ..., numpy.nan, inplace=True)`
- b) Show the summary of the dataframe
- c) Count the **number** and **percentage** of rows with missing in the entire dataset
- d) Count the **number** and **percentage** of missing data in each
- e) **Randomly** drop 10% of the rows that have missing data. Hint: use **any()** and **numpy.random.choice()**
- f) Fill in the missing data with **median** (if the column data type is numeric) or **mode** (if the column data type is not numeric)
- g) Show the summary of the **updated** dataframe
- h) Extract the row with **max** SalePrice and the row with **min** SalePrice
- i) Insert a new column called **Overall**, which is the average of OverallQual and OverallCond.
- j) Update the dataframe with SalePrice used as **index** and **ID** becomes a new column

Lab 01

```
# 1. Missing data could be anything in { NA, n/a, " ", -1, ...}. Read the data and assign the
# missing data in the dataframe as NaN. Hint: df.replace( ..., numpy.nan, inplace=True)
df = pd.read_csv('Mydrive/MyDrive/Lab1_dataset.csv')
list_rep=["NA", "n/a", " ", "-1", "..."]
for x in list_rep:
    df.replace(x, np.nan, inplace=True)
```

```
# 3. Count the number and percentage of rows with missing in the en re dataset
rows_with_nan = df.isnull().any(axis=1).sum()
total_rows = len(df)
percentage_with_nan = (rows_with_nan / total_rows) * 100 if total_rows > 0 else 0
print(rows_with_nan)
print(total_rows)
print(percentage_with_nan)
```

```
93
99
93.93939393939394
```

```
# 2. Show the summary of the dataframe
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99 entries, 0 to 98
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                     99 non-null    int64
1   MSSubClass             99 non-null    int64
2   MSZoning               99 non-null    object
3   LotFrontage           85 non-null    float64
4   LotArea               99 non-null    int64
5   Street                99 non-null    object
6   Alley                 6 non-null     object
7   LotShape              99 non-null    object
8   LandContour           99 non-null    object
9   Utilities             99 non-null    object
10  LotConfig             99 non-null    object
11  Neighborhood          99 non-null    object
12  BldgType              99 non-null    object
13  HouseStyle            95 non-null    object
14  OverallQual           94 non-null    float64
15  OverallCond           95 non-null    float64
16  YearBuilt             99 non-null    int64
17  YearRemodAdd          99 non-null    int64
18  GarageCars            99 non-null    int64
19  SalePrice             99 non-null    int64
dtypes: float64(3), int64(7), object(10)
memory usage: 15.6+ KB
```

Lab 01

```
# 4. Count the number and percentage of missing data in each
```

```
total_cols=len(df.columns)
```

```
nan_count_per_row = df.isna().sum(axis=1)
```

```
percentage_with_nan_in_row =
```

```
(nan_count_per_row / total_cols) * 100 if total_rows > 0 else 0
```

```
print(nan_count_per_row)
```

```
print(percentage_with_nan_in_row)
```

```
0      1
```

```
1      1
```

```
2      1
```

```
3      2
```

```
4      1
```

```
..
```

```
94     1
```

```
95     3
```

```
96     1
```

```
97     1
```

```
98     1
```

```
Length: 99, dtype: int64
```

```
0      5.0
```

```
1      5.0
```

```
2      5.0
```

```
3     10.0
```

```
4      5.0
```

```
...
```

```
94     5.0
```

```
95     15.0
```

```
96     5.0
```

```
97     5.0
```

```
98     5.0
```

```
Length: 99, dtype: float64
```

```
# 5. Randomly drop 10% of the rows that have missing data. Hint: use any() and
```

```
# numpy.random.choice()
```

```
missing_mask = df.isna().any(axis=1)
```

```
missing_rows = df[missing_mask]
```

```
n_to_drop = int(len(missing_rows) * 0.10)
```

```
rows_to_drop = np.random.choice(missing_rows.index, n_to_drop, replace=False)
```

```
df = df.drop(rows_to_drop)
```

```
# 6. Fill in the missing data with median (if the column data type is numeric) or mode (if the  
# column data type is not numeric)
```

```
numeric_cols = df.select_dtypes(include=['number']).columns
```

```
for col in numeric_cols:
```

```
    df[col] = df[col].fillna(df[col].median())
```

```
non_numeric_cols = df.select_dtypes(exclude=['number']).columns
```

```
for col in non_numeric_cols:
```

```
    df[col] = df[col].fillna(df[col].mode()[0])
```

Lab 01

```
# 7. Show the summary of the updated dataframe
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 82 entries, 0 to 98
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                     82 non-null    int64
1   MSSubClass             82 non-null    int64
2   MSZoning               82 non-null    object
3   LotFrontage           82 non-null    float64
4   LotArea               82 non-null    int64
5   Street                82 non-null    object
6   Alley                 82 non-null    object
7   LotShape              82 non-null    object
8   LandContour           82 non-null    object
9   Utilities             82 non-null    object
10  LotConfig             82 non-null    object
11  Neighborhood           82 non-null    object
12  BldgType              82 non-null    object
13  HouseStyle            82 non-null    object
14  OverallQual           82 non-null    float64
15  OverallCond           82 non-null    float64
16  YearBuilt             82 non-null    int64
17  YearRemodAdd          82 non-null    int64
18  GarageCars            82 non-null    int64
19  SalePrice             82 non-null    int64
dtypes: float64(3), int64(7), object(10)
memory usage: 13.5+ KB
```

```
# 8. Extract the row with max SalePrice and the row with min SalePrice
```

```
max_sale_row = df[df['SalePrice'] == df['SalePrice'].max()]
min_sale_row = df[df['SalePrice'] == df['SalePrice'].min()]
print(max_sale_row)
print(min_sale_row)
```

```
   Id  MSSubClass MSZoning  LotFrontage  LotArea Street Alley LotShape \
58  59           60      RL          66.0   13682   Pave  Grvl    IR2
   LandContour Utilities LotConfig Neighborhood BldgType HouseStyle \
58          HLS    AllPub   CulDSac    StoneBr    1Fam    2Story
   OverallQual  OverallCond  YearBuilt  YearRemodAdd  GarageCars  SalePrice
58           10.0           5.0     2006         2006           3     438780
   Id  MSSubClass MSZoning  LotFrontage  LotArea Street Alley LotShape \
30  31           70  C (all)          50.0    8500   Pave  Pave    Reg
   LandContour Utilities LotConfig Neighborhood BldgType HouseStyle \
30          Lvl    AllPub    Inside    IDOTRR    1Fam    2Story
   OverallQual  OverallCond  YearBuilt  YearRemodAdd  GarageCars  SalePrice
30           4.0           4.0     1920         1950           1     40000
```

Lab 01

```
# 9. Insert a new column called Overall, which is the average of OverallQual and  
# OverallCond.
```

```
df['Overall'] = (df['OverallQual'] + df['OverallCond']) / 2
```

```
# 10. Update the dataframe with SalePrice used as index and ID becomes a new column
```

```
df = df.reset_index()
```

```
df = df.set_index('SalePrice')
```

Lab 02

Apply Linear Regression to predict house prices for each of the following cases.

For each case:

1. normalize data using MinMaxScaler()
2. use 80% data for training and 20% for testing
3. write the **Linear Equation** for predicting house prices
4. compute **MSE** score for testing data
5. compute the **coefficient of determination R2 score** for testing data for each case

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

There are 6 cases:

1. Use only area
2. Use only area, bedrooms
3. Use only area, bedrooms, bathrooms
4. Use only area, bedrooms, bathrooms, stories
5. Use only area, bedrooms, bathrooms, stories, parking
6. Use only area, bedrooms, bathrooms, stories, parking, airconditioning (yes->1, no->0)

Summarize MSE, R2 metrics of 6 cases in a Table

Lab 02

```
1 from sklearn.linear_model import LinearRegression
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import MinMaxScaler
4 from sklearn.metrics import mean_squared_error
5
6 # Use only area to predict house price
7 df6 = df[['area', 'bedrooms', 'bathrooms', 'stories', 'parking', 'airconditioning', 'price']]
8 #turn airconditioning into binary variable
9 df6['airconditioning'] = df6['airconditioning'].apply(lambda x: 1 if x == 'yes' else 0)
10
11 # normalize the data using MinMaxScaler
12 scaler = MinMaxScaler()
13 df6_scaled = scaler.fit_transform(df6)
14
15 # split the data into training and testing sets
16 X_df6 = df6_scaled[:, :6] # area, bedrooms, bathrooms, stories, parking, and airconditioning
17 y_df6 = df6_scaled[:, 6] # price
18 X_train_df6, X_test_df6, y_train_df6, y_test_df6 = train_test_split(X_df6, y_df6, test_size=0.2, random_state=42)
19
20 # Write Linear Equation
21 model = LinearRegression()
22 model.fit(X_train_df6, y_train_df6)
23
24 #Predict the price of a house based on its area
25 predicted_prices_scaled = model.predict(X_test_df6)
26 # dataframe to store actual and predicted prices
27 results_df = pd.DataFrame({
28     'Actual Price (scaled)': y_test_df6,
29     'Predicted Price (scaled)': predicted_prices_scaled
30 })
31
32 #Compute MSE
33 y_pred_df6 = model.predict(X_test_df6)
34 mse_df6 = mean_squared_error(y_test_df6, y_pred_df6)
35 print(f'Mean Squared Error: {mse_df6}')
36
37 #Compute coefficient of determination R-squared score
38 r_squared_df6 = model.score(X_test_df6, y_test_df6)
39 print(f'R-squared score: {r_squared_df6}')
40
41 results_df
```

Mean Squared Error: 0.015586746378668016

R-squared score: 0.5886275524061089

	# Actual Price (scaled)	# Predicted Price (scaled)
0	0.19999999999999998	0.3493097543751771
1	0.4242424242424242	0.43594237314563855
2	0.16969696969696965	0.12644324805184412
3	0.40606060606060606	0.26991849083997776
4	0.09090909090909088	0.1362738335220285
5	0.2727272727272727	0.20386174542664118
6	0.30303030303030303	0.37369988961481804
7	0.2418181818181818	0.3025778354852654
8	0.06060606060606058	0.1313000476095501
9	0.13878787878787877	0.10541848243514006

109 rows x 2 cols per page

Lab 02

Liner Regression

$$\hat{y}(w, x) = w_0 + w_1x_1 + \dots + w_px_p$$

```
# Write Linear Equation  
model = LinearRegression()  
model.fit(X_train_df1, y_train_df1)
```

MinMaxScaler

```
# normalize the data using MinMaxScaler  
scaler = MinMaxScaler()  
df1_scaled = scaler.fit_transform(df1)
```

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Lab 02

```
1 # Summarize MSE, R2 metric of 6 cases in the a Table
2 summary_df = pd.DataFrame({
3     'Features': ['area', 'area, bedrooms', 'area, bedrooms, bathrooms',
4                 'area, bedrooms, bathrooms, stories', 'area, bedrooms, bathrooms, stories, parking',
5                 'area, bedrooms, bathrooms, stories, parking, airconditioning'],
6     'MSE': [mse_df1, mse_df2, mse_df3, mse_df4, mse_df5, mse_df6],
7     'R-squared': [r_squared_df1, r_squared_df2, r_squared_df3, r_squared_df4, r_squared_df5, r_squared_df6]
8 })
9 summary_df
```

	Features	# MSE	# R-squared
0	area	0.02755035778765904	0.27287851871974633
1	area, bedrooms	0.024588569145810743	0.3510473817558716
2	area, bedrooms, bathrooms	0.020614609766001796	0.45592991188724397
3	area, bedrooms, bathrooms, stories	0.018423505121883694	0.5137585349037065
4	area, bedrooms, bathrooms, stories,	0.017186496098089366	0.5464062355495862
5	area, bedrooms, bathrooms, stories,	0.015586746378668016	0.5886275524061089

6 rows x 3 cols 10 ▾ per page

⏪ < Page 1 of 1 > ⏩

Lab 02

MSE

$$\text{MSE}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} (y_i - \hat{y}_i)^2.$$

- MSE đo **độ lệch trung bình** giữa giá trị thực và giá trị dự đoán, tính theo đơn vị **bình phương của target (price)**.
- Được coi là hàm đánh giá mất mát (loss function) của mô hình Linear Regression
- MSE càng giảm mô hình dự đoán càng sát với dữ liệu thực tế

R² Score

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- R² là hệ số xác định đo **tỷ lệ phương sai trong biến phụ thuộc y được giải thích bởi mô hình**, không phụ thuộc đơn vị, luôn nằm trong [0,1] với mô hình.
- Với R² càng tăng thì mô hình càng dự đoán đúng với dữ liệu